

Node4All: Learning Node Representation Beyond Datasets

Dooho Lee
KAIST
Daejeon, Republic of Korea
dooho@kaist.ac.kr

Jaemin Yoo
Seoul National University
Seoul, Republic of Korea
jaeminyoo@snu.ac.kr

Abstract

Node representation learning has advanced rapidly, yet most existing methods rely on per-dataset training and hyperparameter tuning. This dataset-specific optimization comes from the difficulty of designing reusable graph models that generalize across diverse graph datasets. In this work, we introduce Node4All, a node representation learner *applicable to arbitrary graph datasets without any dataset-specific optimization*.

Node4All is built on two complementary ideas. At the architectural level, we introduce the Channel Graph Transformer (CGT), which enables a single fixed parameterization to process arbitrary graph datasets. At the learning level, we propose a self-supervised learning based on a series of synthetic graphs. Together, these components enable generalization beyond individual datasets, which is infeasible with existing architectures and learning frameworks. We extensively evaluate Node4All on node classification across 25 benchmarks against 21 baselines, covering both supervised and self-supervised methods. Despite all baselines being trained and optimized for each dataset, a single Node4All, applied uniformly across the datasets, achieves a competitive ranking of 5th among 21 baselines. Moreover, Node4All supports one-shot and in-context learning with an appropriate predictor and outperforms recent graph foundation models (GFM) in these settings. These results demonstrate that Node4All not only achieves reusability across arbitrary graph datasets, but also remains an effective solution in practice. Code and model checkpoints are available in [this link](#).

CCS Concepts

• Computing methodologies → Learning latent representations; Dimensionality reduction and manifold learning.

Keywords

Graph Representation Learning; Channel Graph; Synthetic Graph Generation; Self-supervised Learning; Graph Foundation Models

ACM Reference Format:

Dooho Lee and Jaemin Yoo. 2026. Node4All: Learning Node Representation Beyond Datasets. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD 2026)*, August 9–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770855.3817828>



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
KDD 2026, Jeju Island, Republic of Korea.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817828>

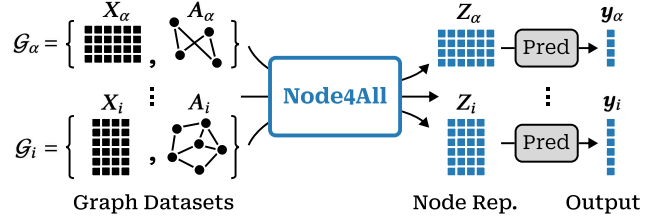


Figure 1: Overview. Node4All is a pretrained node representation learner that generalizes across arbitrary graph datasets.

1 Introduction

A graph is a data structure consisting of a set of nodes V and a set of edges E that connect pairs of nodes. Graphs provide a natural way to describe many real-world systems by representing entities and their relationships, such as social networks, molecules, knowledge graphs, and biological interactions [33, 42, 66]. At the same time, this flexibility makes graphs difficult to process directly: different graphs can have different numbers of nodes $N = |V|$, and their connectivity patterns in E can vary widely, complicating the design of a single processing framework that works across graphs.

Graph representation learning (GRL) aims to address this challenge by mapping a graph to a set of compact node representations $\{z_i\}_{i=1}^N$ in a shared latent space $z_i \in \mathbb{R}^H$ where H denotes the representation dimensionality [4, 35, 37]. These representations provide a standardized interface of graph-structured data and are commonly used as inputs to downstream predictors. As their quality directly influences downstream performance, learning node representations that effectively capture both structural and feature information has remained a central research problem for over a decade.

Early approaches fall into two main paradigms. The first factorizes proximity matrices derived from graph connectivity, including Laplacian Eigenmaps [1], LLE [55], and HOPE [52]. The second learns node embeddings from random-walk co-occurrences using skip-gram-style objectives, including DeepWalk [53], LINE [60], and node2vec [24]. While both paradigms are simple and effective, these methods learn embeddings tied to node identities and therefore require full re-optimization when new nodes are introduced, making the learned representations *node-bound*.

Graph neural networks (GNNs) address this limitation by learning shared message-passing functions that compute node representations from node features and local graph structure, rather than from node identities [3, 13, 25, 64, 67, 68]. This design enables inductive application to unseen nodes and, with advanced architectures and tuned hyperparameters, GNNs achieve strong empirical performance across diverse benchmarks [40, 47, 48]. However, GNNs are typically trained in supervised settings tied to a specific label space, making the learned representations inherently *label-bound* and necessitating re-optimization for each new label set.

Recently, self-supervised learning (SSL) on graphs has been actively explored. SSL-based methods remove the reliance on labels by learning representations through training objectives defined directly on the input graph. Contrastive methods encourage agreement between representations obtained from different augmented views of the same node or graph, such as BGRL [62], MVGRL [27], and CCA-SSG [75]. Generative approaches learn representations by reconstructing masked or perturbed graph structures and node features, including GAE [38], GraphMAE [32], and MaskGAE [44]. The resulting representations are reusable across label spaces and often outperform supervised GNNs.

Despite these significant advances, most graph representation learning methods, from early random-walk-based approaches to recent SSL methods, remain largely *dataset-bound*. They are trained separately on each target dataset and require dataset-specific architectural choices and hyperparameter tuning to achieve strong performance. Furthermore, models trained on one dataset are not typically applicable to graphs with different feature specifications or topologies. As a result, from a user’s perspective, graph learning requires repeating training and tuning for every new dataset, a process that is time-consuming, expertise-intensive, and often requiring labeled data for reliable validation.

In this work, we propose *Node4All*, a single node representation learner that can be applied to arbitrary graph datasets, as illustrated in Figure 1. Node4All imposes no restrictions on graph topology or feature specification and eliminates the need for dataset-specific optimization, including both training and hyperparameter tuning. With Node4All, users can obtain reusable node representations via a single inference. Node4All is composed of two key components that address complementary aspects of the problem:

1. **Architecture.** We propose *Channel Graph Transformer (CGT)*, a novel architecture that can process arbitrary graphs under a single fixed parameterization without any specific tuning.
2. **Learning.** We propose a self-supervised learning approach on synthetic graphs, which are carefully designed to mimic massive real-world graphs, to enable generalization beyond datasets.

We evaluate Node4All on node classification across 25 benchmarks spanning diverse feature specifications and structural properties. We compare against 21 baselines, including both supervised and self-supervised methods, each extensively optimized for individual datasets. Although Node4All is (i) trained without observing any real-world datasets and (ii) applied uniformly across all benchmarks without any modification, it achieves a competitive ranking of 5th among 21 baselines. Moreover, our experiments further show that with an appropriate predictor, Node4All supports one-shot and in-context learning, outperforming recent graph foundation models (GFM) in these settings. These results demonstrate that Node4All not only achieves reusability across arbitrary graph datasets, but also generalizes well and remains effective in practice.

2 Preliminaries

We introduce the notations and background on graph representation learning and GNNs used throughout the paper.

Notations. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph with $N = |\mathcal{V}|$ nodes. Its adjacency matrix is $\mathbf{A} \in \{0, 1\}^{N \times N}$, where $A_{ij} = 1$ if and only if $(i, j) \in \mathcal{E}$. Node features are given by $\mathbf{X} \in \mathbb{R}^{N \times F}$, where

each row $\mathbf{X}_{n,:} \in \mathbb{R}^F$ represents the feature vector of the n -th node with dimensionality F , and each column $\mathbf{X}_{:,f} \in \mathbb{R}^N$ represent the f -th feature channel across all N nodes. Without loss of generality, we represent the graph as $\mathcal{G} = (\mathbf{X}, \mathbf{A})$ throughout this work.

Graph Representation Learning. Given a graph $\mathcal{G} = (\mathbf{X}, \mathbf{A})$ with N nodes, we formalize graph representation learning as learning a function Ψ that maps the graph to H -dimensional node representations $\{\mathbf{z}_i\}_{i=1}^N$, collected as a matrix $\mathbf{Z} = \Psi(\mathbf{X}, \mathbf{A}) \in \mathbb{R}^{N \times H}$:

$$\Psi : \mathbb{R}^{N \times F} \times \{0, 1\}^{N \times N} \rightarrow \mathbb{R}^{N \times H}.$$

Our goal is to learn a representation function Ψ that can process graphs with arbitrary numbers of nodes ($\forall N$) and feature dimensions ($\forall F$) within a single fixed parameterization. The resulting embeddings $\mathbf{Z}$ should jointly encode features and graph structure, serving as sufficient node representations for downstream tasks.

Graph Neural Networks. Given a graph $\mathcal{G} = (\mathbf{X}, \mathbf{A})$ with N nodes, a graph neural network (GNN) iteratively updates node representations $\mathbf{Z}^{(l)} \in \mathbb{R}^{N \times F_l}$, starting from $\mathbf{Z}^{(0)} = \mathbf{X}$, where F_l denotes the representation dimensionality at layer l . Each layer can be written as a message-passing function $\phi^{(l)}$ [23, 34, 69],

$$\mathbf{Z}^{(l)} = \phi^{(l)}(\mathbf{Z}^{(l-1)}, \mathbf{A}) = \text{Comb}^{(l)}\left(\text{Agg}^{(l)}(\mathbf{Z}^{(l-1)}, \mathbf{A}), \mathbf{Z}^{(l-1)}\right), \quad (1)$$

where $\text{Agg}^{(l)}$ aggregates the neighboring representations using a matrix $\tilde{\mathbf{A}} \in \mathbb{R}^{N \times N}$ derived from adjacency matrix $\mathbf{A}$, either statically [25, 39] or dynamically [10, 64], and $\text{Comb}^{(l)}$ integrates the aggregated messages with the previous representations, typically via a learnable linear transformation $\mathbf{W}^{(l)} \in \mathbb{R}^{F_{l-1} \times F_l}$.

3 Limitations of Existing Works

We analyze three common assumptions underlying most existing graph representation learning methods, which limit their applicability and generalization beyond individual datasets.

L1. Fixed Feature Dimensionality. Most existing methods instantiate the representation function Ψ using GNNs. These models encode information through feature transformations parameterized by weight matrices $\mathbf{W} \in \mathbb{R}^{F \times H}$, where F denotes the input feature dimensionality and H the hidden dimension. This design inherently couples the model parameterization to a fixed input dimensionality F . In practice, however, graph datasets exhibit substantial variation in feature dimensionality, ranging from only a few dimensions [54] to several thousand [57]. Consequently, models trained on a specific feature dimensionality cannot be directly transferred to datasets with different dimensionalities $\tilde{F} \neq F$, preventing a single trained model from being applied across datasets.

L2. Optimized on a Specific Dataset. Most graph representation learning methods are trained and optimized on a specific target dataset. This dataset-centric optimization ties the learned representations to dataset-specific regularities, including both structural patterns and feature distributions. As a result, such models often struggle to generalize to nodes or graphs that are rare or differ from those observed during training [43, 51, 71]. While optimization on a target dataset is a natural and effective choice for in-dataset performance, it inherently limits the reuse of learned representations across arbitrary graph datasets.

L3. Specialized to the Target Task. In supervised learning, representations are updated to separate nodes with different labels

while collapsing nodes with the same label. When transferred to a different task with a new label definition, such representations may no longer align with the new semantics, leading to degraded performance. Although SSL removes explicit label supervision, many methods are not fully task-independent in practice. In particular, contrastive approaches rely on the assumption that differently augmented views share the same semantic meaning. This assumption may not hold depending on the task and dataset, and in practice effective augmentations are usually identified through validation performance on the target task. As a result, this indirect guidance ties the learned representations to that task, limiting their generalization to different tasks and datasets.

Together, we identify the obstacles that prevent existing methods from generalizing beyond individual datasets from three aspects of the learning framework: what they train **L1**, where they train **L2**, and how they train **L3**. In Sections 4 and 5, we introduce the components of *Node4All* that address each of these challenges.

4 Processing Arbitrary Graphs

In this section, we introduce channel-wise graph processing and its realization, the *Channel Graph Transformer (CGT)*, which can process graphs with arbitrary feature dimensionality under a single parameterization, thereby addressing **L1**.

4.1 Channel-wise Graph Processing

Our key idea is to treat node features as a collection of independent feature channels rather than as a single vector. By processing each channel with a shared model, the architecture naturally supports arbitrary feature dimensionalities under a single parameterization. To formalize this, we define a *channel graph* as a graph induced by a single feature channel with shared node and edge sets.

Definition 4.1 (Channel Graph). Let $\mathcal{G} = (X, A)$ be a graph whose node features consist of F scalar signals $X = \{X_{:,c} = \mathbf{x}^{(c)} \in \mathbb{R}^N\}_{c=1}^F$. The *channel graph* induced by channel c is defined as

$$\mathcal{G}^{(c)} = (\mathbf{x}^{(c)}, A), \quad c = 1, \dots, F.$$

From this perspective, the graph $\mathcal{G}$ can be regarded as a collection of channel graphs $\{\mathcal{G}^{(c)}\}_{c=1}^F$ that share the same topology. This viewpoint naturally leads to *channel-wise graph processing*, where the same model is applied uniformly to each channel graph.

Definition 4.2 (Channel-wise Graph Processing). Given a graph $\mathcal{G} = \{\mathcal{G}^{(c)}\}_{c=1}^F$, *channel-wise graph processing* defines a representation function Ψ by applying a shared model f to each channel graph and concatenating the results along feature dimension:

$$\begin{aligned} Z &= \Psi(X, A) \triangleq [f(\mathcal{G}^{(1)}) \parallel \dots \parallel f(\mathcal{G}^{(F)})] \\ &= [z^{(1)} \parallel \dots \parallel z^{(F)}] \in \mathbb{R}^{N \times F}, \end{aligned}$$

where the shared model $f : \mathbb{R}^N \times \{0, 1\}^{N \times N} \rightarrow \mathbb{R}^N$ maps each channel graph $\mathcal{G}^{(c)}$ to a scalar node embedding $z^{(c)} \in \mathbb{R}^N$.

This formulation allows f to process graphs with arbitrary feature dimensionality F , thereby addressing **L1**. We next discuss which architecture should be used to realize f .

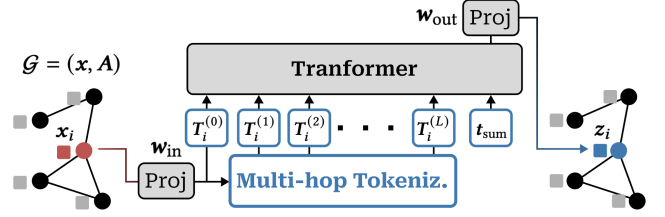


Figure 2: The Channel Graph Transformer (CGT) maps each channel graph to scalar node embeddings via multi-hop tokenization and Transformer-based token readout.

4.2 Channel Graph Transformer (CGT)

A standard GNN is suboptimal for processing channel graphs, as its representational power largely arises from parametric feature transformations that mix multiple feature channels. This capacity disappears if a channel graph is given as input, and GNNs degenerate into repeated propagation of a one-dimensional signal.

To address this limitation, we propose the Channel Graph Transformer (CGT), an architecture specialized for channel graphs. As shown in Figure 2, CGT (i) encodes each node in the channel graph, represented by a scalar feature, into a series of multi-hop tokens, and (ii) uses a Transformer [63] to aggregate the tokens into an updated scalar embedding. This design provides substantially greater expressive capacity for learning on channel graphs than GNNs.

4.2.1 Multi-hop Tokenization. A Transformer requires tokens with a fixed dimensionality that carry meaningful information on their own, such as image patches for computer vision [14] or word tokens for language [63]. In graphs, however, a node feature alone does not encode structural information, since such information is defined through interactions with its neighborhood. CGT constructs standardized and structurally informative tokens through *coordinate-wise multi-hop aggregation*.

Given a channel graph $\mathcal{G} = (x, A)$, we first map the scalar node signal to an initial d -dimensional token

$$T^{(0)} = x w_{\text{in}} \in \mathbb{R}^{N \times d}, \quad w_{\text{in}} \in \mathbb{R}^{1 \times d}.$$

We then assign each coordinate of the token a distinct receptive-field size. At each CGT layer l , the i -th coordinate is propagated by exactly i hops over the graph, and the resulting representations are combined to form a new d -dimensional token:

$$T^{(l)} = g^{(l)} \left(\left[\hat{A}^0 T_{:,0}^{(l-1)} \parallel \hat{A}^1 T_{:,1}^{(l-1)} \parallel \dots \parallel \hat{A}^{d-1} T_{:,d-1}^{(l-1)} \right] \right) \in \mathbb{R}^{N \times d},$$

where $\hat{A}$ denotes the symmetrically normalized adjacency matrix. The mixing function $g^{(l)} : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times d}$ is implemented using a lightweight multi-layer perceptron (MLP) for simplicity.

Our tokenization method ensures that each token has a consistent dimensionality d while embedding rich neighborhood information of each node from local to global scales. Repeating this for L layers yields a series of tokens $T^{(0)}, T^{(1)}, \dots, T^{(L)}$, which serve as the input sequence to the Transformer.

4.2.2 Transformer-based Token Readout. CGT then aggregates the token sequence of each node into a single representation using a Transformer. To perform this aggregation, we prepend a learnable

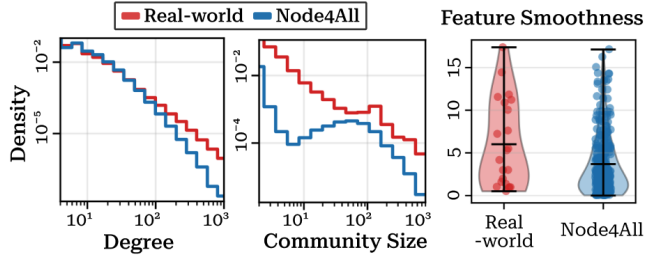


Figure 3: Comparison between 25 real-world graph datasets and 300 Node4All synthetic graphs. Degree and community-size (identified by the Louvain algorithm) distributions show similar trends between real and synthetic graphs. Per-dataset average feature smoothness also spans a comparable range between synthetic and real-world datasets.

summary token $\mathbf{t}_{\text{sum}} \in \mathbb{R}^d$ to the token sequence of each node i :

$$\mathbf{S}_i^{(0)} = [\mathbf{t}_{\text{sum}}, \mathbf{T}_i^{(0)}, \mathbf{T}_i^{(1)}, \dots, \mathbf{T}_i^{(L)}] \in \mathbb{R}^{(L+2) \times d}, \quad i = 1, \dots, N.$$

We then apply an R -layer Transformer to this sequence. Through self-attention, it attends over the entire token set and aggregates the information into $\mathbf{t}_{\text{sum}}$. We take the output representation of $\mathbf{t}_{\text{sum}}$ at layer R and project it to a scalar embedding:

$$f(\mathcal{G})_i = \mathbf{S}_{i,0}^{(R)\top} \mathbf{w}_{\text{out}} \in \mathbb{R}, \quad \mathbf{w}_{\text{out}} \in \mathbb{R}^d.$$

Together, CGT realizes the function f required for channel-wise graph processing, while providing sufficient expressive capacity to capture structural information in the channel graph.

5 Learning for Arbitrary Graphs

While our CGT enables the processing of arbitrary graphs, learning representations that generalize across datasets remains challenging, which relates to limitations L2 and L3. To address these limitations, we propose a new SSL framework based on synthetic graphs.

5.1 Synthetic Graph Generation

Most graph models are trained on only one or a small number of real-world datasets and are fitted to dataset-specific patterns (L2). In the language and vision domains, this issue is often mitigated by training a model on massive collections of real-world datasets. However, achieving such large-scale coverage is difficult for graph models due to the lack of semantically aligned datasets.

To address this challenge, we train the model on synthetic graphs instead of limited real-world datasets, exposing it to a wide range of graph distributions and encouraging it to focus on structural and feature patterns that persist across datasets. Given that many possible design choices are available for synthetic graph generation, we keep the design simple to avoid introducing unnecessary inductive biases while covering a wide range of graph distributions.

5.1.1 Overview. We define a synthetic graph generator as a probability distribution over graphs $\mathcal{G} = (\mathbf{X}, \mathbf{A})$, denoted by $p(\mathcal{G})$ and parameterized by latent variables $\mathbf{z}$:

$$p(\mathcal{G}) = \int p(\mathbf{z}) p(\mathbf{A} | \mathbf{z}) p(\mathbf{X} | \mathbf{A}, \mathbf{z}) d\mathbf{z}.$$

A graph $\mathcal{G} \sim p(\mathcal{G})$ is generated by a three-stage process: (i) sampling latent variables $\mathbf{z}$, (ii) generating the graph structure $\mathbf{A}$ conditioned on $\mathbf{z}$, and (iii) generating node features $\mathbf{X}$ conditioned on both $\mathbf{A}$ and $\mathbf{z}$. We next describe the structure and feature generators, as well as the latent parameters that control them.

5.1.2 Structure Generation. We generate graph structures using a Chung–Lu–style random graph model [12]. The structure generator is controlled by two graph-level latent variables: the expected average degree $\bar{d}$ and a degree bias parameter σ , which determines the degree heterogeneity across nodes.

Given the number of nodes N and parameters $(\bar{d}, \sigma)$, we assign each node $u \in \{1, \dots, N\}$ an expected degree w_u via

$$\alpha_u \sim \mathcal{N}(-\frac{1}{2}\sigma^2, \sigma^2), \quad w_u = \bar{d} \exp(\alpha_u).$$

This approach preserves the expected average degree across nodes, i.e., $\mathbb{E}[w_u] = \bar{d}$, while introducing heterogeneity of individual nodes. When $\sigma \approx 0$, the graph is close to regular, whereas larger σ yields increasingly heavy-tailed degree distributions.

Given the sampled expected degrees $\{w_u\}_{u=1}^N$, we then sample an undirected adjacency matrix $\mathbf{A} \in \{0, 1\}^{N \times N}$ by independently drawing each unordered node pair $1 \leq u < v \leq N$ with

$$A_{uv} \sim \text{Bernoulli}(p_{uv}), \quad p_{uv} = \min\left(1, \frac{w_u w_v}{W}\right), \quad W = \sum_{t=1}^N w_t.$$

Under this construction, the realized degree d_u of each node satisfies $\mathbb{E}[d_u | \{w\}] \approx w_u$. Consequently, the graph-level average degree concentrates around $\frac{1}{N} \sum_{u=1}^N d_u \approx \bar{d}$. Overall, this procedure generates diverse graph structures while controlling sparsity through $\bar{d}$ and degree heterogeneity through σ , yielding degree distributions similar to those of real-world graph datasets (see Figure 3).

5.1.3 Feature Generation. Our feature generation pipeline is inspired by recent tabular foundation models [29], but is adapted to graphs in two key ways: (i) by explicitly controlling the intrinsic dimensionality of the feature matrix, and (ii) by modeling correlations between node features and graph structure through propagation.

Controlling intrinsic dimensionality is motivated by the observation that many real-world datasets lie on low-dimensional semantic manifolds despite being represented in high-dimensional feature spaces. This property is particularly important for graphs, where lower intrinsic dimensionality increases the relative contribution of structural information in downstream tasks.

To capture this effect, we introduce a rank parameter $\alpha \in (0, 1]$, which determines the number of semantic factors $\bar{F} = \lceil \alpha F \rceil$ for a given feature dimension F . We sample a semantic matrix $\mathbf{S} \in \mathbb{R}^{N \times \bar{F}}$ and map it to the feature space via linear mixing,

$$\mathbf{X} = \mathbf{S}\mathbf{W} \in \mathbb{R}^{N \times F}, \quad \mathbf{W} \in \mathbb{R}^{\bar{F} \times F}.$$

We initialize $\mathbf{W}$ with orthonormal rows so that the effective rank [56] of $\mathbf{X}$ is approximately $\bar{F}$, thereby controlling intrinsic dimensionality through α . Feature diversity arises from sampling $\mathbf{S}$ and $\mathbf{W}$, while varying α induces diversity in intrinsic dimensionality, allowing the generator to produce a range of realistic low-rank feature structures observed in real-world datasets.

However, since these features are generated independently of the graph structure, they exhibit no inherent correlation with the graph. In particular, as shown theoretically in Appendix A, random

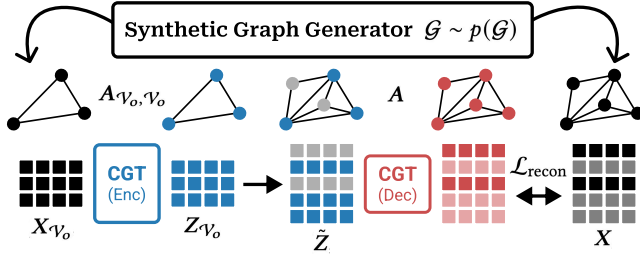


Figure 4: Node4All trains CGT via masked autoencoding on a stream of synthetic graphs to learn node representations that generalize to arbitrary real-world graph datasets.

assignment of node features degenerates graph-aligned variations in expectation. To counteract this effect, we apply stochastic propagation that explicitly amplifies graph-aligned variations:

$$X \leftarrow \left((1 + \beta\alpha)I - \alpha\hat{A} \right)^S X, \quad \beta \in \{0, 1\}, \alpha \in [0, 1],$$

where $\hat{A}$ denotes the symmetrically normalized adjacency matrix. The operator is controlled by three latent variables: the propagation depth S , the mixing strength α , and a mode selector β . When $\beta = 0$, the operator reduces to $I - \alpha\hat{A}$, emphasizing differences between a node and its neighbors. When $\beta = 1$, it becomes $I + \alpha(I - \hat{A})$, which amplifies these differences while retaining the original node signal. By sampling S , α , and β , the propagation introduces diverse patterns of graph-aligned features. As a result, the generated features span a range of smoothness comparable to that observed in real-world graph datasets, as shown in Figure 3.

5.2 Masked Autoencoder with DropNode

Having specified what to train (CGT) and where to train it (synthetic graphs), we now describe how we train it. We adopt *masked node-feature reconstruction*, following GraphMAE [31, 32], which learns representations by reconstructing masked node features.

This self-supervised objective relies on a weak and broadly applicable assumption: node attributes are at least partially predictable from their surrounding context. If this assumption were violated, it would imply that nodes in the graph are largely independent of one another, calling into question whether the graph structure encodes meaningful relational information at all. Such scenarios are uncommon in practical graph datasets, making masked reconstruction a robust objective that mitigates task-specific bias (L3).

However, directly applying GraphMAE is not ideal in our setting. Unlike prior work that assumes a single dataset with a fixed feature space, our training data consists of multiple synthetic graphs, each with potentially different feature spaces. Thus, replacing masked nodes with a single shared learnable token, e.g., [MASK], as done in GraphMAE, can lead to inconsistent semantics across graphs and introduce misleading shortcuts during training.

We therefore adopt a DropNode-style [18, 73] masked autoencoder, which drops nodes instead of replacing them with a mask token. Given a synthetic graph $\mathcal{G} = (X, A)$ with a node set $\mathcal{V}$, we randomly select a subset of nodes to drop, $\tilde{\mathcal{V}} \subset \mathcal{V}$, and denote the remaining nodes by $\mathcal{V}_o = \mathcal{V} \setminus \tilde{\mathcal{V}}$. The encoder then operates on the induced subgraph formed by removing dropped nodes and their

incident edges, producing embeddings for the remaining nodes:

$$Z_{\mathcal{V}_o} = f_{\text{enc}}(X_{\mathcal{V}_o}, A_{\mathcal{V}_o, \mathcal{V}_o}), \quad \tilde{Z}_i = \begin{cases} Z_i, & i \in \mathcal{V}_o, \\ 0, & i \in \tilde{\mathcal{V}}. \end{cases}$$

For reconstruction, we feed the decoder the full graph structure with a node representation matrix $\tilde{Z} \in \mathbb{R}^{N \times D}$, which contains the encoded representations for retained nodes and the zero vectors for dropped nodes. We optimize the autoencoder f_{enc} and f_{dec} using a cosine reconstruction loss over the dropped nodes:

$$\mathcal{L}_{\text{recon}} = \frac{1}{|\tilde{\mathcal{V}}|} \sum_{i \in \tilde{\mathcal{V}}} \left(1 - \cos(f_{\text{dec}}(\tilde{Z}, A)_i, X_i) \right).$$

Node4All. Finally, we introduce *Node4All*, a node representation learner that integrates the three components: Channel Graph Transformer (CGT; Section 4.2), synthetic graph generation (Section 5.1), and masked autoencoding (Section 5.2), as illustrated in Figure 4. Node4All uses CGT as both the encoder f_{enc} and decoder f_{dec} , enabling it to process graphs with arbitrary feature spaces. During training, a synthetic graph generator produces diverse graphs, and the model is trained via masked autoencoding by reconstructing dropped node features from their surrounding graph context. By repeatedly performing this reconstruction across heterogeneous graph structures and feature spaces, Node4All learns representations that capture generalizable patterns rather than dataset-specific ones, allowing node representation learning to generalize beyond individual datasets toward arbitrary graph datasets.

6 Related Works

Node4All is a node representation learner that shares the goal with graph representation learning methods, which we review in detail in Section 1. Here, we focus on two closely related lines of work: (1) graph transformers (GTs), which are architecturally related, and (2) graph foundation models (GFM), which similarly aim to generalize across multiple graph datasets within a single model.

6.1 Graph Transformers

GTs are a class of architectures that model graph-structured data using Transformers. Many GTs apply self-attention across nodes and augment attention with topology-aware positional encodings [15, 72]. While effective, their attention mechanism incurs quadratic complexity with respect to the number of nodes. To improve scalability, several GTs instead represent each node as a sequence of topology-aware tokens and apply self-attention within the sequence. For example, NAGphormer constructs tokens by aggregating features from different hop distances [6], VCR-Graphormer builds token sequences using informative context nodes identified through personalized PageRank and virtual connections [21], and PolyFormer constructs tokens using polynomial graph filters that capture structural information at multiple scales [49].

While CGT also applies self-attention over node-specific token sequences, it differs fundamentally in how tokens are constructed. Existing methods operate on node feature vectors, where each token is derived from a rich multi-dimensional representation. In contrast, CGT operates on channel graphs, where each token originates from a scalar-valued feature channel. Applying the same aggregation operator to all feature coordinates would therefore yield

highly redundant representations, as every coordinate encodes the same signal at the same neighborhood scale. CGT addresses this issue through coordinate-wise multi-hop tokenization, assigning a distinct receptive field to each coordinate. Consequently, each token captures structural information from multiple neighborhood scales, yielding a richer representation of the channel signal.

6.2 Graph Foundation Models

We categorize existing GFM approaches into three directions based on how broadly they generalize across datasets.

Text-space GFMs The first direction unifies multiple graph datasets by operating in a common text space. These methods leverage pretrained large language models (LLMs) to embed raw text features [5, 45] or to make predictions directly within them [8, 61, 70]. They have shown promising results in handling task heterogeneity, co-training on multiple datasets, transfer learning, and few-shot learning [8, 17, 28, 45]. However, many graph datasets contain numerical or categorical attributes, and converting them into text is often infeasible or relies on dataset-specific templates [9].

Feature-aligned GFMs The second direction extends generalization to graphs with non-textual attributes. To address feature heterogeneity across datasets (L1), these approaches project node features into a shared representation space via singular value decomposition with alignment modules [74, 76] or learnable patching mechanisms [59], and have shown promising results in co-training and few-shot adaptation. However, these methods typically require per-dataset fine-tuning when adapting to new graphs and are less effective when sufficient labels are available [59, 74, 76].

Fully Inductive GFMs The third direction studies fully inductive GFMs, which operate on arbitrary graph datasets without training on the target data [77]. GraphAny achieves full inductiveness by attending over a fixed set of least-mean-square linear predictors derived from features propagated at different scales. TS-GNNs [20] further generalizes this prediction framework by enforcing equivariance to node and feature permutations and invariance to labels. NodePFN [11] pursues a related goal by extending tabular foundation models, such as TabPFN [29, 30], to graph-structured inputs. More recently, Recurrent GVT (RGVT) [41] achieves cross-graph transfer by learning over a graph-filter-induced view space rather than the original node feature space.

Relationship with GFMs Node4All differs from existing GFMs in two key aspects. First, we target full inductive applicability across arbitrary graph datasets without any dataset-specific modification. This is challenging for text-space GFMs due to their reliance on textual attributes and for feature-aligned GFMs due to their dependence on per-dataset fine-tuning. While NodePFN and RGVT do not require training on the target dataset, they still rely on per-dataset preprocessing and recurrent depth selection, respectively. Second, unlike most GFMs that conduct task-specific predictions, Node4All creates reusable representations. This enables (i) a single inference to produce embeddings that can be cached and reused across different label spaces, and (ii) flexible choices of downstream predictors. In our experiments, we show that both one-shot settings of feature-aligned GFMs and in-context prediction of fully inductive GFMs can be realized, and often improved upon, by selecting an appropriate predictor on top of Node4All representations.

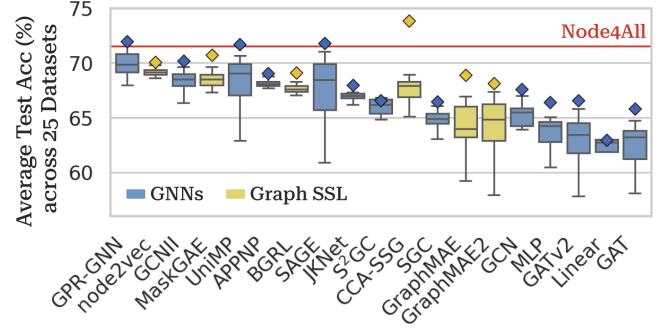


Figure 5: Box plots show the distribution of average test accuracies (%) across hyperparameter configurations for each method. Diamonds denote the performance of each method under Tuned-HP. The red line is Node4All, which exhibits no variation since it requires no dataset-specific tuning.

7 Main Experiments

We evaluate the practical effectiveness of Node4All through a series of experiments comparing it with existing methods across multiple settings. Specifically, we investigate the following questions:

- Q1. Can a single fixed model (Node4All) make competitive performance compared to per-dataset optimized methods?
- Q2. How effective are Node4All representations in one-shot learning compared to feature-aligned GFMs?
- Q3. How effective are Node4All representations in in-context learning compared to fully inductive GFMs?

7.1 Against Per-Dataset Optimized Methods

Node4All offers clear practical advantages, including (i) applicability to arbitrary graph datasets and (ii) the elimination of dataset-specific training and hyperparameter tuning. However, these benefits are only meaningful if performance remains competitive. If too much accuracy is sacrificed, users may instead prefer the cost of per-dataset optimization. We therefore evaluate whether a single fixed Node4All model can compete with conventional methods that are trained and optimized separately for each dataset.

Setup. We compare Node4All against 21 baselines, including linear and MLPs predictors, node2vec [24] with concatenated initial features, 12 supervised GNNs, and 6 graph SSL methods. The supervised baselines include GCN [39], GraphSAGE [25], APPNP [22], JKNet [69], GAT [64], SGC [67], GCNII [7], GATv2 [3], S²GC [78], UniMP [58], and GPR-GNN [10]. The self-supervised baselines include contrastive approaches (BGRL [62] and CCA-SSG [75]) and generative methods (GAE [38], GraphMAE [32], GraphMAE2 [31], and MaskGAE [44]). For all representation learning methods, including Node4All, we use the same one-hidden-layer MLP predictor to isolate the quality of the learned representations.

We use 25 node classification benchmarks covering diverse domains, feature specifications, and structural properties, with label homophily ranging from 0.06 to 0.84 and average degree from 4.03 to 89.28. Dataset details are provided in Appendix D. Public data splits are used when available; otherwise, following [77], we sample 20 nodes per class for training and evenly split the remaining nodes into validation and test sets. For datasets with random splits, we

select hyperparameters based on 5 independent splits, while test accuracy is reported as the mean and the standard deviation over a broader range of 10 splits.

Results. Due to the large number of benchmarks and baselines, we report a summary in Figure 5; full results are available on the project website.¹ We evaluate baselines under two settings, **Fixed-HP** and **Tuned-HP**, to disentangle the effects of (i) per-dataset training and (ii) per-dataset hyperparameter tuning. All baselines are trained separately on each target dataset, whereas Node4All is pretrained once on synthetic graphs and applied to all datasets as is, without any *dataset-specific training or hyperparameter tuning*.

In **Fixed-HP**, the results are shown as the box plots in Figure 5. Each method uses a single hyperparameter configuration selected to maximize its average validation accuracy across all datasets. This setting isolates the effect of per-dataset training and tests whether a method admits a generally effective configuration that transfers across datasets. Under this setting, Node4All achieves the highest average accuracy among all 21 baselines. At the level of individual datasets, this result indicates that the knowledge captured through synthetic pretraining can match, and even surpass, the knowledge obtained through direct supervision or self-supervised learning on target graphs. At the cross-dataset level, the results show that a single Node4All model can generalize across diverse graph datasets by learning from large collections of synthetic graphs, outperforming methods trained separately on each dataset. Overall, these findings suggest that when extensive per-dataset hyperparameter tuning is not affordable, Node4All provides a strong and reliable alternative to conventional per-dataset training.

In **Tuned-HP**, the results are visualized as the diamond markers in Figure 5. Hyperparameters are selected independently for each dataset, allowing each method to benefit from dataset-specific hyperparameter choices. Since Node4All does not perform any per-dataset hyperparameter tuning, its performance remains the same as in **Fixed-HP**. Under this setting, graph SSL methods improve their average accuracy by 1.92%, and supervised GNNs baselines by 1.11%, while Node4All’s rank shifts from 1st to 5th among 21 baselines. This highlights a key property of existing methods: their strongest results typically emerge only after extensive per-dataset tuning and training, and a single broad configuration is often insufficient. The cost of this specialization is reflected in strong hyperparameter sensitivity, as illustrated in Figure 5, which shows substantial variation in average performance across hyperparameter configurations. In contrast, Node4All employs a single fixed model, requires no per-dataset optimization, and yet remains competitive even against heavily tuned baseline methods, making it a more robust and practically favorable choice.

7.2 One-Shot Learning

One-shot learning is a common evaluation setting for GFM, where only a single labeled example per class is provided. It is designed to assess whether knowledge pretrained on other datasets can be effectively adapted to a new graph dataset under minimal supervision. We use this setting to evaluate the effectiveness of Node4All representations under extremely limited supervision and compare

Table 1: One-shot learning performance (%) compared with feature-aligned GFMs across 5 datasets.

Method	Cora	Citeseer	Pubmed	AmzPhoto	AmzComp
GCN	29.53 $\pm$ 7.56	26.29 $\pm$ 6.50	23.32 $\pm$ 11.56	26.96 $\pm$ 12.94	24.40 $\pm$ 5.62
GraphPrompt	28.26 $\pm$ 12.68	32.51 $\pm$ 8.73	47.47 $\pm$ 9.15	48.11 $\pm$ 9.89	42.82 $\pm$ 11.67
GPF	32.17 $\pm$ 6.56	36.79 $\pm$ 7.70	41.28 $\pm$ 8.14	47.47 $\pm$ 8.19	35.75 $\pm$ 7.12
Hassani	33.35 $\pm$ 6.93	33.66 $\pm$ 7.24	39.87 $\pm$ 8.16	48.48 $\pm$ 7.07	39.99 $\pm$ 7.91
GCOPE	35.62 $\pm$ 11.93	38.33 $\pm$ 9.28	45.38 $\pm$ 9.87	52.87 $\pm$ 9.19	45.65 $\pm$ 10.69
SAMGPT	47.80 $\pm$ 11.88	36.38 $\pm$ 9.10	50.65 $\pm$ 8.93	58.71 $\pm$ 8.69	48.22 $\pm$ 8.17
Ridge Reg.	30.54 $\pm$ 10.63	39.83 $\pm$ 8.15	52.42 $\pm$ 8.09	52.42 $\pm$ 8.09	38.58 $\pm$ 8.87
+ Node4All	51.79 $\pm$ 12.99	44.08 $\pm$ 8.94	38.76 $\pm$ 10.51	58.97 $\pm$ 9.69	49.72 $\pm$ 10.64

them against recent feature-aligned GFMs, which explicitly aim to support cross-dataset adaptation in this setting.

Setup. We follow the evaluation protocol of SAMGPT [74] and use their reported numbers on five datasets. Baselines include supervised GCN [39], feature-aligned GFMs (GCOPE [76], SAMGPT [74]), graph cross-domain adaptation model (Hassani [26]), and graph prompting frameworks (GPF [16], GraphPrompt [46]). Node4All is applied to all datasets without any dataset-specific modification, as in the main experiment. To adapt the node representations to the labels with only one-shot supervision, we first reduce their dimensionality to 32 using truncated SVD and then train a light-weight ridge regressor with a regularization strength $\alpha = 10.0$. This procedure is applied uniformly across all datasets.

Results. We report the accuracy in Table 1. Node4All outperforms GCOPE and SAMGPT, which are feature-aligned GFMs, on 4 out of 5 datasets, as well as other baselines. These results demonstrate that Node4All remain effective even under extreme supervision scarcity. Importantly, while the GFM baselines rely on co-training across multiple real-world datasets, Node4All is trained solely on synthetic graphs. These findings support our claim that training on well-designed synthetic graphs can generalize more effectively than training on a limited collection of real-world datasets.

7.3 In-Context Learning

In-context learning refers to making predictions without updating model parameters, instead using a small set of labeled examples as context. For GFMs, this setting evaluates whether pretrained knowledge from other datasets can be effectively applied to unseen datasets by conditioning on the provided context. We use this evaluation to assess whether Node4All representations support in-context adaptation and compare them against fully inductive GFMs, which are designed explicitly for in-context learning on graphs.

Setup. We compare against three recent fully inductive GFMs: GraphAny [77], TSNet [20], and NodePFN [11]. We follow the shared evaluation protocol when evaluating Node4All, while baseline results are taken directly from the original papers.

Since Node4All does not natively support in-context learning, we pair it with TabPFN [29, 30], a tabular foundation model, as an in-context predictor. Node representations produced by Node4All are provided to TabPFN together with the labeled nodes as context. This Node4All-TabPFN pipeline enables in-context predictions on arbitrary graphs in a single inference pass, without any training or dataset-specific modification, thereby forming a fully inductive

¹https://github.com/dooho00/node4all/blob/main/docs/full_exp_results.pdf

Table 2: In-context learning performance (%) compared with fully inductive GFM across 11 datasets. The last column reports the average rank (lower is better), with **best and second-best results highlighted for each dataset.**

Method	Actor	Amazon Photo	Amazon Computer	Cornell	DBLP	Deezer	Mine-sweeper	Pubmed	Tolokers	Texas	Wisconsin	Avg-Rank
GraphAny	29.51 $\pm$ 0.55	90.64 $\pm$ 0.82	83.04 $\pm$ 1.24	66.49 $\pm$ 1.48	66.49 $\pm$ 1.48	52.13 $\pm$ 3.02	80.46 $\pm$ 0.15	77.46 $\pm$ 0.30	78.24 $\pm$ 0.03	73.52 $\pm$ 2.96	71.77 $\pm$ 5.98	3.90
TS-GAT	28.80 $\pm$ 2.12	89.66 $\pm$ 1.23	80.23 $\pm$ 2.44	71.35 $\pm$ 4.52	64.30 $\pm$ 3.75	52.88 $\pm$ 2.86	80.72$\pm$0.78	75.46 $\pm$ 0.86	78.01 $\pm$ 0.23	74.05 $\pm$ 4.10	65.88 $\pm$ 9.86	4.27
TS-Mean	28.09 $\pm$ 0.93	90.18 $\pm$ 1.30	81.37 $\pm$ 1.25	68.65 $\pm$ 2.42	66.42 $\pm$ 3.65	52.31 $\pm$ 2.52	<u>80.68$\pm$0.38</u>	74.98 $\pm$ 0.56	78.12 $\pm$ 0.09	73.51 $\pm$ 4.01	61.18 $\pm$ 11.38	4.63
NodePFN	32.99 $\pm$ 1.09	90.53 $\pm$ 0.13	81.42 $\pm$ 0.48	71.89 $\pm$ 2.76	<u>74.71$\pm$0.39</u>	53.45 $\pm$ 0.65	80.66 $\pm$ 0.25	<u>78.00$\pm$0.24</u>	<u>78.61$\pm$0.06</u>	76.22 $\pm$ 7.53	79.22 $\pm$ 6.97	<u>2.54</u>
TabPFN	32.93 $\pm$ 1.71	80.78 $\pm$ 1.63	67.02 $\pm$ 2.00	74.05$\pm$5.57	53.46 $\pm$ 2.53	55.10$\pm$2.48	80.00 $\pm$ 0.00	72.20 $\pm$ 0.00	78.36 $\pm$ 0.24	82.08$\pm$5.16	86.08$\pm$3.97	3.63
+ Node4All	33.18$\pm$1.15	91.70$\pm$0.65	83.59$\pm$0.73	66.76 $\pm$ 7.05	77.67$\pm$1.54	55.06 $\pm$ 1.30	80.36 $\pm$ 0.36	80.80$\pm$0.00	81.15$\pm$0.61	76.88 $\pm$ 5.40	82.35 $\pm$ 3.62	2.00

GFM. Due to current limitations of TabPFN on feature dimensionality and the number of context samples, we evaluate this setup on 11 datasets that satisfy these constraints.

Results. We report the results in Table 2. Node4All achieves the best average rank of 2.0 and the top performance on 6 datasets. These results show that Node4All representations are highly effective for in-context learning, transforming the structure-agnostic TabPFN into a graph foundation model and outperforming state-of-the-art fully inductive GFMs. Notably, NodePFN, the second-best method, requires per-dataset data preprocessing, including the PCA dimensionality and propagation depth, making our strong performance particularly remarkable given that it requires no dataset-specific modification. Moreover, Node4All with TabPFN generally outperforms Node4All with the MLP predictor reported in the main experiment, highlighting the promise of this combination.

8 Additional Analysis

In this section, we present further analyses of Node4All, examining its computational complexity, representational behavior via visualization, and the effects of its core design choices.

8.1 Complexity Analysis

Theoretical. The computational cost of Node4All is dominated by CGT, which includes multi-hop tokenization followed by Transformer pooling. Let d be the token dimension, $(L + 1)$ the token sequence length, and R the number of layers. The computational cost decomposes into multi-hop tokenization $O(L(|E|d^2 + Nd^2))$, Transformer attention $O(RN(L + 1)^2d)$, and feed-forward layers $O(RN(L + 1)d^2)$, all of which are linear with the number of nodes N and edges $|E|$. Since this cost is incurred per channel graph, CGT scales linearly with the input feature dimensionality F .

Empirical. On an NVIDIA RTX A6000, training Node4All takes about 10 minutes for 500 iterations over 500 synthetic graphs, and inference across 25 benchmark graphs takes 672 seconds. The main bottleneck is that CGT processes all F feature channels separately, without compressing them into a hidden dimension H as in standard GNNs, leading to higher inference time on datasets with large feature dimensionality, such as Coauthor Physics (238 s), FullCora (103 s), and BlogCatalog (90 s) with even $H > 8000$. Nevertheless, runtime scales linearly with F and remains practical, with inference time of several minutes. Moreover, Node4All offers two efficiency advantages: (i) representations can be cached and reused, and (ii) no dataset-specific hyperparameter search is required, which often dominates the overall cost of existing methods.

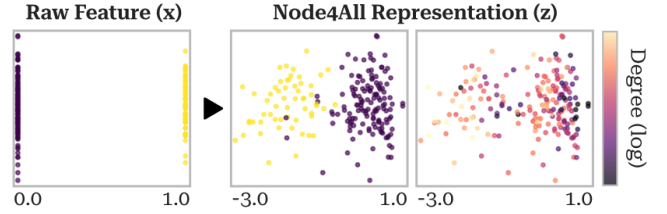


Figure 6: Scatter plots of raw features (left) and Node4All representations (middle, right) for the first feature channel of Amazon Photo, with nodes colored by degree (right).

8.2 Visualization

While recent fully inductive GFMs can operate across arbitrary datasets, it remains unclear what kind of knowledge enables such broad generalization. In this section, we use visualizations to examine how Node4All behaves and to explain why this mechanism supports generalization across datasets.

Channel-wise Processing. Node4All processes each feature channel independently, which allows us to directly inspect how an individual channel changes before and after graph-aware encoding. In Figure 6, each node appears as a point, with its scalar feature value on the horizontal axis, while the vertical axis is included for visualization. For binary features, many nodes initially share identical values (0 or 1; left). After encoding, Node4All maps these nodes to distinct representations while preserving the original two-group separation (middle), and the resulting ordering further correlates with node degree (right). These results show how Node4All injects structural information into each node group that shares their features. We observe the same behavior consistently for categorical and continuous features, as shown in Appendix C.

Representation Space. We next visualize the full node representations using t-SNE [50]. When colored by the first principal component of the raw features, the overall feature-driven structure is largely preserved in the learned representations. When colored by node degree, clear structural patterns emerge in the Node4All representation space that are not visible in the raw features. Together, these observations suggest that the channel-wise structural effects introduced by Node4All accumulate coherently in the final representation space. Importantly, this mechanism—distinguishing nodes based on graph structure while preserving original feature information—relies on generic structural cues rather than dataset-specific semantics, which helps explain why Node4All generalizes effectively across diverse graph datasets.

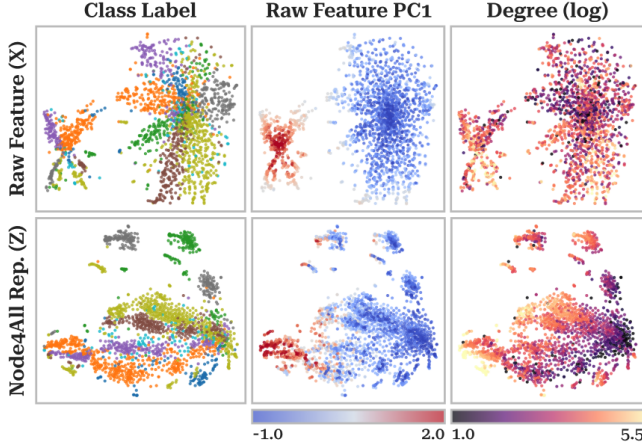


Figure 7: t-SNE visualizations of raw features and Node4All representations on Amazon Photo, colored by class labels (left), raw-feature PC1 (middle), and node degree (right).

8.3 Ablation Study

Due to space limitations, detailed ablation results are provided in Appendix B. Replacing the backbone architecture, CGT with a GCN reduces average accuracy to 37.80%, far below the Node4All model (71.55%), highlighting the necessity of CGT for channel-wise graph processing. Training on real-world datasets instead of synthetic graphs yields lower performance, with OGBN-Arxiv achieving the best average accuracy (69.94%), comparable to mid-ranked baselines (9th of 21). This result indicates that Node4All’s architecture and masked autoencoding form an effective framework for cross-dataset representation learning, while synthetic graph pretraining drives strong generalization to arbitrary graph datasets.

9 Discussion

In this section, we discuss the inductive bias of Node4All, as well as its current limitations and directions for future research.

9.1 Inductive Bias

CGT processes each feature channel independently, enabling it to handle arbitrary feature dimensionalities. Beyond this flexibility, the channel-wise design promotes transferability across datasets by avoiding the explicit modeling of cross-feature interactions. While individual feature channels often carry transferable semantic information, the ways in which features interact to predict labels can vary substantially across datasets and label spaces. Capturing such interactions during pretraining may therefore cause representations to become specialized to particular datasets and label spaces, limiting their ability to generalize across datasets. Consistent with this view, explicitly modeling cross-feature interactions through linear self-attention reduces the average performance of Node4All across the 25 benchmark datasets to 70.78%. This result supports our design choice of preserving channel identities while learning structural patterns during pretraining and delegating the modeling of cross-feature interactions to downstream predictors.

Our learning framework further reinforces this inductive bias. The feature reconstruction objective encourages the preservation of feature semantics, while training on diverse synthetic graphs

promotes generalization across varying graph structures and feature spaces. Together, these design choices enable Node4All to learn transferable representations across diverse graph datasets.

9.2 Limitations and Future Work

Synthetic graph generation. The primary goal of this work is to establish the feasibility of dataset-agnostic graph representation learning. Consequently, our synthetic graph generation framework remains relatively simple. An important direction for future work is to explore richer graph generation strategies that capture a broader range of structural patterns. For example, stochastic block model-based generators [36] could provide more realistic community structures, while motif-aware generation strategies [2] could introduce higher-order patterns commonly observed in real-world graphs. By generating graphs whose structural properties more closely resemble those of real-world datasets, such approaches may yield more transferable and robust node representations.

Beyond node classification. This work focuses exclusively on node classification tasks. Extending dataset-agnostic representation learning to graph-level tasks involving small graphs, such as molecular property prediction, as well as edge-level tasks, is an important direction toward more general graph representation learners. Supporting these prediction settings may require architectural adaptations and pretraining objectives tailored to different prediction levels. For example, edge-level tasks may benefit from objectives that explicitly capture pairwise relationships and local connectivity patterns, while graph-level tasks may require mechanisms that better aggregate graph-wide information.

10 Conclusion

In this work, we introduce Node4All, a single node representation learner for arbitrary real-world graph datasets, without requiring any dataset-specific training or hyperparameter tuning. Node4All differs from existing methods in both its architecture and learning framework. Architecturally, we propose the Channel Graph Transformer (CGT), a novel channel-wise graph processing architecture that accommodates heterogeneous feature dimensionalities across datasets within a single fixed model. In terms of learning, unlike conventional graph representation learning methods that are optimized separately for each target dataset, Node4All is trained once, entirely on synthetic graphs. Through carefully designed synthetic graph generation that spans a diverse range of graphs, Node4All learns representations that generalizable across a wide range of graph datasets. Empirically, a single Node4All model achieves competitive performance against per-dataset optimized methods and even outperforms recent graph foundation models in one-shot and in-context learning. Overall, Node4All advances node representation learning beyond dataset-bounds.

Acknowledgments

This work was partly supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2024-00341425 and RS-2024-00406985), “Advanced GPU Utilization Support Program” funded by the Government of the Republic of Korea (Ministry of Science and ICT), and the New Faculty Startup Fund from Seoul National University. Jaemin Yoo is the corresponding author.

References

- [1] William N Anderson Jr and Thomas D Morley. 1985. Eigenvalues of the Laplacian of a graph. *Linear and multilinear algebra* 18, 2 (1985), 141–145.
- [2] Austin R Benson, David F Gleich, and Jure Leskovec. 2016. Higher-order organization of complex networks. *Science* 353, 6295 (2016), 163–166.
- [3] Shaked Brody, Uri Alon, and Eran Yahav. 2021. How attentive are graph attention networks? *arXiv preprint arXiv:2105.14491* (2021).
- [4] Fenxiao Chen, Yun-Cheng Wang, Bin Wang, and C-C Jay Kuo. 2020. Graph representation learning: a survey. *APSIPA Transactions on Signal and Information Processing* 9 (2020), e15.
- [5] Haibo Chen, Xin Wang, Zeyang Zhang, Haoyang Li, Ling Feng, and Wenwu Zhu. 2025. Autogfm: Automated graph foundation model with adaptive architecture customization. In *Forty-second International Conference on Machine Learning*.
- [6] Jinsong Chen, Kaiyuan Gao, Gaichao Li, and Kun He. 2023. NAGphormer: A Tokenized Graph Transformer for Node Classification in Large Graphs. In *The Eleventh International Conference on Learning Representations*.
- [7] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. 2020. Simple and deep graph convolutional networks. In *International conference on machine learning*. PMLR, 1725–1735.
- [8] Runjin Chen, Tong Zhao, Ajay Kumar Jaiswal, Neil Shah, and Zhangyang Wang. 2024. LLaGA: Large Language and Graph Assistant. In *International Conference on Machine Learning*. PMLR, 7809–7823.
- [9] Zhiqiang Chen, Hongxin Mao, Jun Liu, Yizhou Song, Bing Li, Wei Jin, Yizhou Ren, Yifan Hou, Pan Li, Chao Wang, Carl Yang, and Jiliang Tang. 2024. Text-space graph foundation models: Comprehensive benchmarks and new insights. In *Advances in Neural Information Processing Systems*, Vol. 37. 7464–7492.
- [10] Eli Chien, Jianhao Peng, Pan Li, and Olga Milenkovic. 2021. Adaptive Universal Generalized PageRank Graph Neural Network. In *International Conference on Learning Representations*.
- [11] Jeongwhan Choi, Jongwoo Kim, Woosung Kang, and Noseong Park. 2026. Learning Posterior Predictive Distributions for Node Classification from Synthetic Graph Priors. In *The Fourteenth International Conference on Learning Representations*.
- [12] Fan Chung and Linyuan Lu. 2002. Connected components in random graphs with given expected degree sequences. *Annals of combinatorics* 6, 2 (2002), 125–145.
- [13] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. *Advances in neural information processing systems* 29 (2016).
- [14] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*.
- [15] Vijay Prakash Dwivedi and Xavier Bresson. 2020. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699* (2020).
- [16] Taoran Fang, Yunchao Zhang, Yang Yang, Chunping Wang, and Lei Chen. 2023. Universal prompt tuning for graph neural networks. *Advances in Neural Information Processing Systems* 36 (2023), 52464–52489.
- [17] Yi Fang, Dongzhe Fan, Sirui Ding, Ninghao Liu, and Qiaoyu Tan. 2025. UniGLM: Training One Unified Language Model for Text-Attributed Graphs Embedding. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining*. 973–981.
- [18] Wenzheng Feng, Jie Zhang, Yuxiao Dong, Yu Han, Huanbo Luan, Qian Xu, Qiang Yang, Evgeny Kharlamov, and Jie Tang. 2020. Graph random neural networks for semi-supervised learning on graphs. *Advances in neural information processing systems* 33 (2020), 22092–22103.
- [19] Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428* (2019).
- [20] Ben Finkelshtein, İsmail İlkan Ceylan, Michael Bronstein, and Ron Levie. 2025. Equivariance Everywhere All At Once: A Recipe for Graph Foundation Models.
- [21] Dongqi Fu, Zhigang Hua, Yan Xie, Jin Fang, Si Zhang, Kaan Sancak, Hao Wu, Andrey Malevich, Jingrui He, and Bo Long. 2024. VCR-Graphormer: A Mini-batch Graph Transformer via Virtual Connections. In *The Twelfth International Conference on Learning Representations*.
- [22] Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. 2019. Combining Neural Networks with Personalized PageRank for Classification on Graphs. In *International Conference on Learning Representations*.
- [23] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *International conference on machine learning*. PMLR, 1263–1272.
- [24] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864.
- [25] Will Hamilton, Zitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [26] Kaveh Hassani. 2022. Cross-domain few-shot graph classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 6856–6864.
- [27] Kaveh Hassani and Amir Hosein Khasahmadi. 2020. Contrastive multi-view representation learning on graphs. In *International conference on machine learning*. PMLR, 4116–4126.
- [28] Yufei He, Yifan Sui, Xiangnan He, and Bryan Hooi. 2025. Unigraph: Learning a unified cross-domain foundation model for text-attributed graphs. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, Volume 1. 448–459.
- [29] Noah Hollmann, Samuel Müller, Katharina Eggenberger, and Frank Hutter. 2022. TabPFN: A transformer that solves small tabular classification problems in a second. *arXiv preprint arXiv:2207.01848* (2022).
- [30] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeyer, and Frank Hutter. 2025. Accurate predictions on small data with a tabular foundation model. *Nature* 637, 8045 (2025), 319–326.
- [31] Zhenyu Hou, Yufei He, Yukuo Cen, Xiao Liu, Yuxiao Dong, Evgeny Kharlamov, and Jie Tang. 2023. Graphmae2: A decoding-enhanced masked self-supervised graph learner. In *Proceedings of the ACM web conference 2023*. 737–746.
- [32] Zhenyu Hou, Xiao Liu, Yukuo Cen, Yuxiao Dong, Hongxia Yang, Chunjie Wang, and Jie Tang. 2022. Graphmae: Self-supervised masked graph autoencoders. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 594–604.
- [33] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.
- [34] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. 2020. Strategies for Pre-training Graph Neural Networks. In *International Conference on Learning Representations*.
- [35] Wei Ju, Zheng Fang, Yiyang Gu, Zequn Liu, Qingqing Long, Ziyue Qiao, Yifang Qin, Jianhao Shen, Fang Sun, Zhiping Xiao, et al. 2024. A comprehensive survey on deep graph representation learning. *Neural Networks* 173 (2024), 106207.
- [36] Brian Karrer and Mark EJ Newman. 2011. Stochastic blockmodels and community structure in networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 83, 1 (2011), 016107.
- [37] Shima Khoshraftar and Aijun An. 2024. A survey on graph representation learning methods. *ACM Transactions on Intelligent Systems and Technology* 15, 1 (2024), 1–55.
- [38] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016).
- [39] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- [40] Dooho Lee, Myeong Kong, Sagad Hamid, Cheonwoo Lee, and Jaemin Yoo. 2025. Aggregation Buffer: Revisiting DropEdge with a New Parameter Block. In *International Conference on Machine Learning*. PMLR, 33181–33204.
- [41] Dooho Lee, Myeong Kong, Minh Jeong, and Jaemin Yoo. 2026. View Space: Learning Representation across Arbitrary Graphs. *arXiv:2512.11561*
- [42] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [43] Haoyang Li, Xin Wang, Ziwei Zhang, and Wenwu Zhu. 2025. Out-of-distribution generalization on graphs: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
- [44] Jintang Li, Ruofan Wu, Wangbin Sun, Liang Chen, Sheng Tian, Liang Zhu, Changhua Meng, Zibin Zheng, and Weiqiang Wang. 2023. What’s behind the mask: Understanding masked graph modeling for graph autoencoders. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1268–1279.
- [45] Hao Liu, Jiarui Feng, Lecheng Kong, Ningyue Liang, Dacheng Tao, Yixin Chen, and Muhan Zhang. 2024. One For All: Towards Training One Graph Model For All Classification Tasks. In *The Twelfth International Conference on Learning Representations*.
- [46] Zemin Liu, Xingtong Yu, Yuan Fang, and Xinming Zhang. 2023. Graphprompt: Unifying pre-training and downstream tasks for graph neural networks. In *Proceedings of the ACM web conference 2023*. 417–428.
- [47] Yuankai Luo, Lei Shi, and Xiao-Ming Wu. 2024. Classic gnns are strong baselines: Reassessing gnns for node classification. *Advances in Neural Information Processing Systems* 37 (2024), 97650–97669.
- [48] Yuankai Luo, Lei Shi, and Xiao-Ming Wu. 2025. Can Classic GNNs Be Strong Baselines for Graph-level Tasks? Simple Architectures Meet Excellence. In *Forty-second International Conference on Machine Learning*.
- [49] Jiahong Ma, Mingguo He, and Zhewei Wei. 2024. Polyformer: Scalable node-wise filters via polynomial graph transformer. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*. 2118–2129.
- [50] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, Nov (2008), 2579–2605.

- [51] Haitao Mao, Zhikai Chen, Wei Jin, Haoyu Han, Yao Ma, Tong Zhao, Neil Shah, and Jiliang Tang. 2023. Demystifying structural disparity in graph neural networks: Can one size fit all? *Advances in neural information processing systems* 36 (2023), 37013–37067.
- [52] Mingdong Ou, Peng Cui, Jian Pei, Ziwei Zhang, and Wenwu Zhu. 2016. Asymmetric transitivity preserving graph embedding. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 1105–1114.
- [53] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 701–710.
- [54] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. 2023. A critical look at the evaluation of GNNs under heterophily: Are we really making progress?. In *The Eleventh International Conference on Learning Representations*.
- [55] Sam T Roweis and Lawrence K Saul. 2000. Nonlinear dimensionality reduction by locally linear embedding. *science* 290, 5500 (2000), 2323–2326.
- [56] Olivier Roy and Martin Vetterli. 2007. The effective rank: A measure of effective dimensionality. In *2007 15th European signal processing conference*. IEEE, 606–610.
- [57] Olexandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of graph neural network evaluation. *arXiv preprint arXiv:1811.05868* (2018).
- [58] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjing Wang, and Yu Sun. 2021. Masked Label Prediction: Unified Message Passing Model for Semi-Supervised Classification. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 1548–1554.
- [59] Yifan Sun, Yizhou Yang, Xiaoyu Feng, Zhe Wang, Haoran Zhong, Chen Wang, and Lei Chen. 2025. Handling feature heterogeneity with learnable graph patches. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Volume 1*. 1313–1324.
- [60] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. 2015. Line: Large-scale information network embedding. In *Proceedings of the 24th international conference on world wide web*. 1067–1077.
- [61] Jiabin Tang, Yuhao Yang, Wei Wei, Lei Shi, Lixin Su, Suqi Cheng, Dawei Yin, and Chao Huang. 2024. Graphgpt: Graph instruction tuning for large language models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 491–500.
- [62] Shantanu Thakoor, Corentin Tallec, Mohammad Gheshlaghi Azar, Rémi Munos, Petar Veličković, and Michal Valko. 2021. Bootstrapped representation learning on graphs. In *ICLR 2021 workshop on geometrical and topological representation learning*.
- [63] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [64] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- [65] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, et al. 2019. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315* (2019).
- [66] Quan Wang, Zhendong Mao, Bin Wang, and Li Guo. 2017. Knowledge graph embedding: A survey of approaches and applications. *IEEE transactions on knowledge and data engineering* 29, 12 (2017), 2724–2743.
- [67] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *International conference on machine learning*. Pmlr, 6861–6871.
- [68] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations*.
- [69] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning*. PMLR, 5453–5462.
- [70] Ruosong Ye, Caiqi Zhang, Runhui Wang, Shuyuan Xu, and Yongfeng Zhang. 2024. Language is all a graph needs. In *Findings of the association for computational linguistics: EACL 2024*. 1955–1973.
- [71] Gilad Yehudai, Ethan Fetaya, Eli Meir, Gal Chechik, and Haggai Maron. 2021. From local structures to size generalization in graph neural networks. In *International Conference on Machine Learning*. PMLR, 11975–11986.
- [72] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do transformers really perform badly for graph representation? *Advances in neural information processing systems* 34 (2021), 28877–28888.
- [73] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *Advances in neural information processing systems* 33 (2020), 5812–5823.
- [74] Xinke Yu, Zeyu Gong, Chen Zhou, Yuxiao Fang, and Hongyang Zhang. 2025. SAMGPT: Text-free graph foundation model for multi-domain pre-training and cross-domain adaptation. In *Proceedings of the ACM on Web Conference 2025*. 1142–1153.
- [75] Hengrui Zhang, Qitian Wu, Junchi Yan, David Wipf, and Philip S Yu. 2021. From canonical correlation analysis to self-supervised graph neural networks. *Advances in Neural Information Processing Systems* 34 (2021), 76–89.
- [76] Hao Zhao, An Chen, Xiaoqian Sun, Hong Cheng, and Jiajun Li. 2024. All in one and one for all: A simple yet effective method towards cross-domain graph pretraining. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4443–4454.
- [77] Jianan Zhao, Zhaocheng Zhu, Mikhail Galkin, Hesham Mostafa, Michael M Bronstein, and Jian Tang. 2025. Fully-inductive Node Classification on Arbitrary Graphs. In *The Thirteenth International Conference on Learning Representations*.
- [78] Hao Zhu and Piotr Koniusz. 2021. Simple Spectral Graph Convolution. In *International Conference on Learning Representations*.

A Limitations of Random Node Assignment

A key challenge in synthetic graph generation is producing node features that reflect the underlying graph structure. A natural approach is to assign features randomly and independently to each node. However, because this process ignores the graph topology, the resulting features contain little information about structural differences between neighboring nodes.

This limitation can be characterized spectrally. High-frequency graph components capture localized variations induced by the graph structure, yet random feature assignment contributes only limited energy to these components. The following lemma formalizes this observation and motivates our use of random propagation to inject graph-dependent structure into the generated features.

LEMMA A.1. *Let $X \in \mathbb{R}^{N \times F}$ be node features generated independently of the graph structure, and let $A \in \mathbb{R}^{N \times N}$ denote the adjacency matrix. Then, in expectation, at most a fraction of k/N of the total feature energy lies in any k -dimensional high-frequency eigenspace of the graph Laplacian.*

PROOF. Let $L = I - D^{-1/2}AD^{-1/2}$ and let P_H be an orthogonal projector onto a k -dimensional high-frequency eigenspace of L . We generate $X = SW$, where $W \in \mathbb{R}^{\tilde{F} \times F}$ has orthonormal rows ($WW^\top = I_{\tilde{F}}$), and for each factor $j \in [\tilde{F}]$ we sample

$$\mu_j \sim \mathcal{N}(0, 1), \quad z_j \sim \mathcal{N}(0, 1), \quad \sigma_j = |z_j \mu_j|,$$

then draw node-wise entries independently as

$$S_{u,j} \mid (\mu_j, \sigma_j) \sim \mathcal{N}(\mu_j, \sigma_j^2), \quad u \in [N].$$

Since $WW^\top = I_{\tilde{F}}$, we have $XX^\top = SS^\top$. Moreover, for $u \neq v$,

$$\mathbb{E}[S_{u,j}S_{v,j}] = \mathbb{E}[\mu_j^2], \quad \mathbb{E}[S_{u,j}^2] = \mathbb{E}[\sigma_j^2] + \mathbb{E}[\mu_j^2],$$

hence

$$\mathbb{E}[XX^\top] = \mathbb{E}[SS^\top] = \tilde{F} \mathbb{E}[\sigma^2] I_N + \tilde{F} \mathbb{E}[\mu^2] \mathbf{1}\mathbf{1}^\top.$$

Therefore,

$$\mathbb{E}[\|P_H X\|_F^2] = \text{tr}(P_H \mathbb{E}[XX^\top]) = \tilde{F} \mathbb{E}[\sigma^2] \text{tr}(P_H) + \tilde{F} \mathbb{E}[\mu^2] \mathbf{1}^\top P_H \mathbf{1}.$$

Using $\text{tr}(P_H) = k$ and $0 \preceq P_H \preceq I$ (so $\mathbf{1}^\top P_H \mathbf{1} \leq \mathbf{1}^\top I \mathbf{1} = N$), we obtain

$$\mathbb{E}[\|P_H X\|_F^2] \leq \tilde{F} \mathbb{E}[\sigma^2] k + \tilde{F} \mathbb{E}[\mu^2] N.$$

Also,

$$\mathbb{E}[\|X\|_F^2] = \text{tr}(\mathbb{E}[XX^\top]) = \tilde{F} \mathbb{E}[\sigma^2] N + \tilde{F} \mathbb{E}[\mu^2] N^2.$$

Table 3: Ablation study on the CGT.

Modification	Total Avg. (25)
Use GAT instead CGT	37.80 $\pm$ 1.29
Use GCN instead CGT	37.61 $\pm$ 0.92
Use GAT instead Multi-hop Tokenization	68.92 $\pm$ 1.34
Use GCN instead Multi-hop Tokenization	69.85 $\pm$ 1.48
Use MLP instead Transformer Readout	68.14 $\pm$ 2.34
Node4All	71.55$\pm$1.38

Table 4: Average test accuracy (%) across 25 datasets for CGT trained on real-world datasets with masked autoencoding.

Dataset	# Nodes	Total Avg. (25)	Rank @ Baselines
Wisconsin	251	60.39 $\pm$ 3.58	20
Cora	2,708	61.79 $\pm$ 3.80	20
Roman Empire	22,662	55.32 $\pm$ 1.78	20
Questions	48,921	69.58 $\pm$ 1.60	9
OGBN-Arxiv	169,343	69.94 $\pm$ 1.48	9
OGBN-Products	2,449,029	68.82 $\pm$ 1.36	10
Node4All	-	71.55$\pm$1.38	5

Hence for $k \geq 1$,

$$\frac{\mathbb{E}[\|P_H X\|_F^2]}{\mathbb{E}[\|X\|_F^2]} \leq \frac{\bar{F} \mathbb{E}[\sigma^2] k + \bar{F} \mathbb{E}[\mu^2] N}{\bar{F} \mathbb{E}[\sigma^2] N + \bar{F} \mathbb{E}[\mu^2] N^2} \leq \frac{k}{N},$$

which proves the claim. $\square$

B Ablation Studies

While we provide a summary of the ablation study in Section 8.3, we present detailed results and analyses here.

Channel Graph Transformer (CGT). CGT is designed to process a *channel graph*, where each node is associated with a single scalar feature. As a baseline, we apply standard GNNs architectures (GCN and GAT) by lifting scalar features to a d -dimensional space, performing message passing, and projecting them back to scalars. As shown in Table 3, this results in severe performance degradation, with average accuracies of 37.80% and 37.61%, respectively, even below a linear classifier (62.92%). These results confirm that learning on channel graphs requires a specialized architecture such as CGT.

CGT consists of two components: multi-hop tokenization and Transformer-based token readout (Section 4.2). Since removing either component makes the model ill-defined, we instead evaluate alternative designs. First, we replace dimension-wise tokenization with shared aggregation using GCN or GAT as token generators. This modification reduces performance to 69.85% and 68.92%, respectively, though the degradation is less severe than using GNNs backbones alone. Similarly, replacing Transformer-based aggregation with MLP pooling also degrades performance, but again to a lesser extent than using GNNs alone. Overall, these results highlight that the tokenization–aggregation paradigm is essential for learning on channel graphs, with the proposed CGT design achieving the best performance.

Training on Real-world Datasets. While Node4All is trained on synthetic graphs to achieve broad generalization, training on

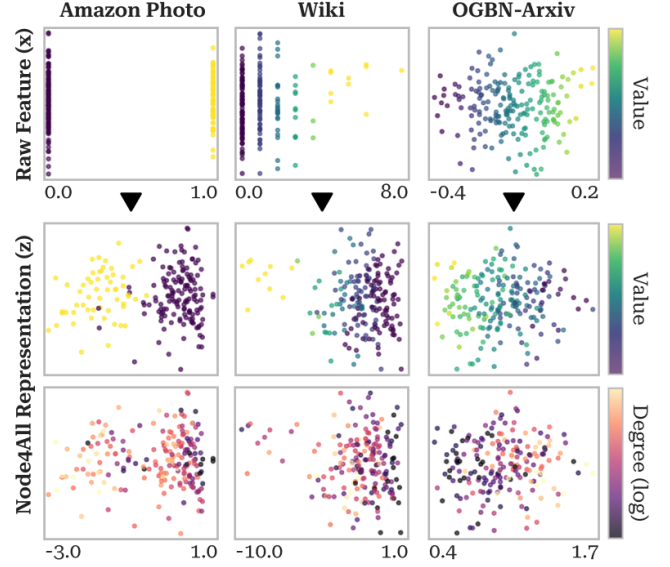


Figure 8: Scatter plots of raw features (top) and Node4All representations (bottom) for the first feature channel on Amazon Photo, Wiki, and OGBN-Arxiv, representing binary, categorical, and continuous features.

real-world datasets is also possible. We evaluate CGT models trained on six real-world datasets (Table 4) and find that all of them underperform the synthetic-pretrained Node4All, highlighting the advantage of using synthetic graphs on pretraining.

In details, CGT fails to learn meaningful representations on small datasets (e.g., Wisconsin, Cora, and Roman Empire), but achieves mid-ranked performance on larger datasets (e.g., Questions, OGBN-Arxiv, and OGBN-Products). However, larger size alone does not guarantee better results: despite being much larger than OGBN-Arxiv, OGBN-Products does not yield improved performance. These observations suggest that effective training depends more on structural and feature diversity than on dataset size alone. Overall, these results show that pretraining on synthetic graph provides a controllable and effective way to induce diverse training signals.

C Additional Visualization

As described in Section 8.2, we visualize how channel values change before and after processing by CGT. Here, we provide additional visualizations on datasets with different feature types—binary, categorical, and continuous—as shown in Figure 8.

D Dataset Details

Our dataset selection and splitting strategy largely follows prior work [77]. We obtain datasets from PyG [19], DGL [65], and the heterophilous graph datasets released by Yandex Research.² All graphs are treated as undirected. For the Chameleon and Squirrel datasets, we use the filtered versions provided in this repository, as the original datasets have been reported to contain issues [54]. Detailed statistics for each dataset are reported in project website.³

²<https://github.com/yandex-research/heterophilous-graphs>

³https://github.com/dooho00/node4all/blob/main/docs/dataset_stats.pdf